EMERGENCY RAPID RESPONSE SYSTEM (ERRS)

Technical Report

Module: COMSM0122 – Fundamentals of System Security

Coursework: Engineering and Securing a National Emergency Response Platform (NERP)

Group Number : 16

Group Members :

1. Darshan
2. Jafar Ayaz Naqvi
3. Xinyi Huang
4. Yan Wang

Submission Details

University: University of Bristol

Submission Date: 6 November 2026

Word Count:

EXECUTIVE SUMMARY

This report describes the design, development and security testing of the Emergency Rapid Response System (ERRS). Our group developed ERRS as a prototype for the National Emergency Response Platform (NERP). The main goal of this project is to help emergency services manage incidents and share information safely. The system also aims to protect sensitive data and keep emergency operations reliable.

Our group built the system using Flask and SQLite. The system allows public users to report incidents and staff members to manage emergency responses. It also supports incident assessment, emergency dispatch, resource allocation and audit records. We added several security controls to protect the system. These include an Access Control Matrix, the Bell-LaPadula model and the Biba model. We also used information classification and access rules to control how different users can view and manage data.

We also tested three consensus protocols: Byzantine Fault Tolerance (BFT), Paxos and Raft. We used Python simulations to test how these protocols respond to attacks and failures. We carried out nine experiments, including key compromise, state loss and network partitioning. The results showed that the simulated protocols blocked unsafe operations in the tested cases.

The project also has some limits. The regional nodes run as simulations in one Python process. The SQLite database does not support real multi-region replication. The system also needs better data anonymisation and secure deletion. Overall, ERRS shows how different security methods can work together in an emergency response system. However, the system needs more development and testing before it can be used in a real national emergency response service.

INTRODUCTION

Emergency response systems are important for public safety. Police officers, ambulance teams, firefighters and government agencies use these systems to share information and manage emergencies. During a major incident, these teams need accurate information to make quick decisions. However, cyber-attacks, system failures and unauthorised access can affect their work. These problems can delay emergency services and put people's safety at risk.

The National Emergency Response Platform (NERP) aims to help different emergency services work together across the country. The system needs to protect sensitive information and keep important services available. It also needs to make sure that the information is correct and reliable. These goals can sometimes conflict. For example, emergency workers may need quick access to important information, but some records contain private or restricted data. The system must protect this data while allowing staff to complete their tasks.

Our group developed the Emergency Rapid Response System (ERRS) to study these security problems. We built the prototype using Flask, SQLite and Python. The system allows public users to report emergencies. It also allows authorised staff to review incidents, manage dispatch requests and assign resources. We added user authentication, access control, data classification and audit functions to improve security.

We also studied three distributed consensus protocols: Byzantine Fault Tolerance (BFT), Paxos and Raft. We used Python simulations to test how these protocols behave when attacks or failures occur. These tests helped us understand why the protocols need certain conditions to work safely.

This report has five main parts. Part A explains the system architecture and possible security threats. Part B discusses the consensus protocols and the results of our security experiments. Part C explains how the system classifies and manages data. Part D describes the access control rules and security models. Finally, Part E discusses security decisions, remaining risks and whether the system is suitable for real-world use.

Part A：Architecture and Threat Modelling

A1. Distributed Architecture Design

Figure 1 shows the proposed NERP architecture. The team uses Flask and SQLite to build the ERRS prototype. The application uses separate modules within one Flask system. It also uses three simulated regions to test distributed consensus

The system provides two portals. Public users submit incident reports and check their status. Staff use a separate portal to review incidents and manage emergency responses. The authentication module checks user identities and sessions. The policy engine then uses ACM, Bell-LaPadula, Biba and context checks to control access.

The Operational Domain manages incidents, assessment, dispatcher review, dispatch, resources and analytics. For example, the assessment module suggests a response, but a dispatcher must review the report before sending service requests. The Restricted Domain manages intelligence, data classification, audit records and controlled exports. The cross-domain gateway checks and filters information shared between the domains.

The architecture uses four trust boundaries. TB1 protects external access, while TB2 controls access between internal modules. TB3 protects database operations, and TB4 checks messages between simulated nodes. These boundaries use identity checks, access rules, input validation and signed messages.

The system keeps five original JSON datasets as source data. SQLite stores application records. North, Central and South act as simulated replica nodes. Raft supports incidents and dispatch, Paxos supports resource allocation, and BFT supports auditing and intelligence distribution. Each protocol checks its quorum before committing changes. If quorum is lost, the system stops new protected commits. Integrity checks and protected backups support recovery.

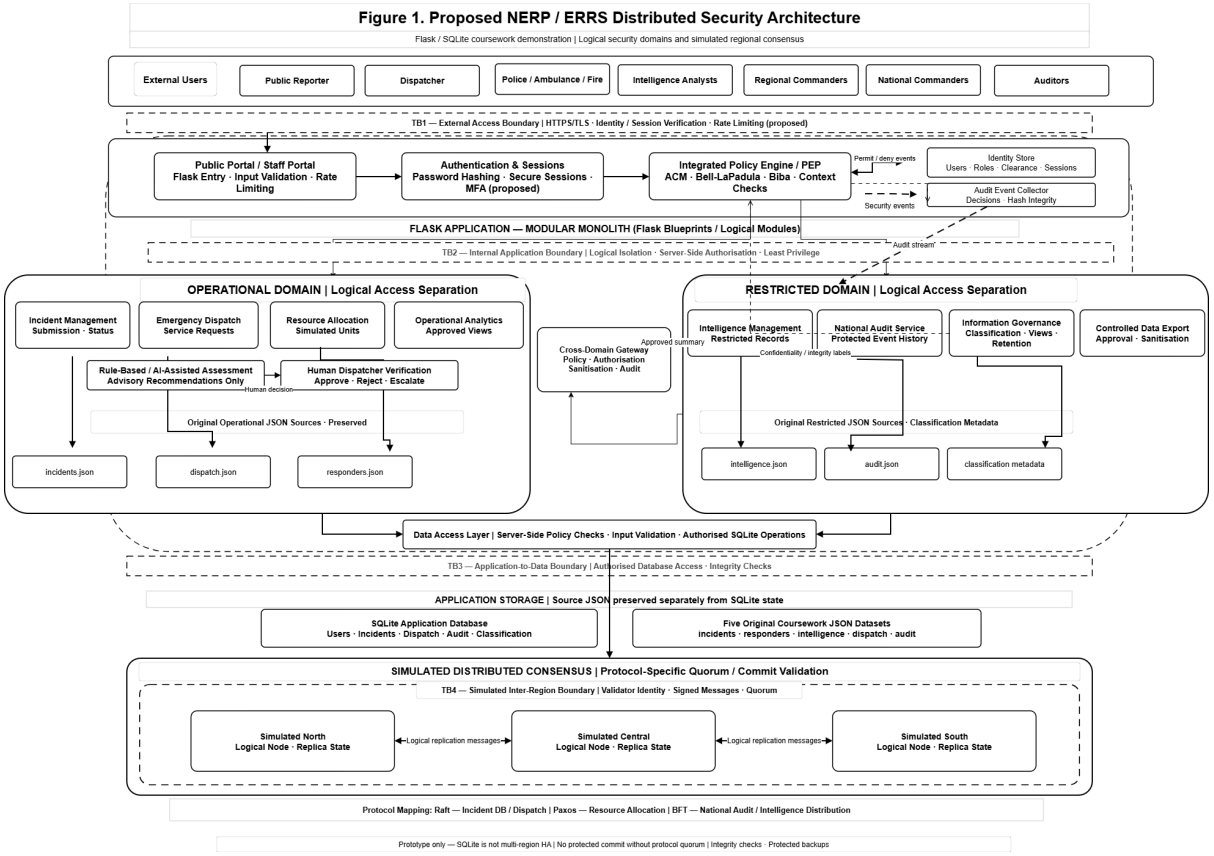


Figure 1: Proposed NERP/ERRS Distributed Security Architecture.

A2 — Threat Model

Figure 2 shows eight main threats to NERP. The table identifies the assets at risk, attack paths, possible results and security controls. These threats affect public access, staff accounts, application modules and stored data.

First, a denial-of-service attack can flood the public portal with requests. This can delay new incident reports and emergency dispatch. An attacker may also steal staff passwords or session tokens to access restricted information. A staff member may misuse their permissions to change incident records or export intelligence. The system uses access checks, secure sessions and audit logs to reduce these risks. Rate limiting provides another proposed control.

Next, malware can enter a Flask module and change incident or dispatch data. A harmful Python package can also affect several modules because they share one application. Input validation and limited write permissions help protect the records. The assessment module provides advice, but a dispatcher must approve service requests. This step helps prevent false incident details from leading directly to resource allocation. Package checks and controlled updates also reduce software risks.

Finally, host failures and network partitions can affect system availability. The team uses simulated regional nodes to test these failures. If the nodes lose quorum, the system stops new protected commits. Data corruption can also damage SQLite records or audit history. Integrity checks, audit verification and protected backups help find problems and support recovery. Stolen accounts, harmful software and long outages remain possible risks.

Figure 2. NERP / ERRS Architecture-Based Threat Model

PART A2 | Eight required threats linked to assets, attack paths, impacts and controls in Figure 1

SCOPE: Public / Staff Portals → TB1 → Flask Modules & Domains (TB2) → SQLite / Source JSON (TB3) → Simulated Nodes (TB4)

ID	Threat	Assets at Risk	Attack Path / Failure	Consequences	Architectural Mitigations	A1 Component / Boundary
T1	**Denial of Service**	Public Portal; Flask Entry; Dispatch	An attacker floods the reporting endpoint or dispatch requests.	Public reports and emergency service requests are delayed.	Proposed rate limiting; backend validation; request monitoring.	TB1 / Portals / Operational
T2	**Credential Theft**	Staff Sessions; Identity Store; Intelligence	An attacker steals a staff password or session token and logs in.	Restricted records are exposed or incident data is changed.	Secure sessions; password hashing; ACM / BLP / Biba / context checks; proposed MFA.	TB1–TB2 / Auth / Policy
T3	**Insider Abuse**	Intelligence; Incident History; Audit	A staff member misuses export rights or changes an incident status.	Private data leaks; incident history or audit trust is weakened.	Least privilege; context checks; controlled export; audit decisions.	Restricted / Gateway / Audit
T4	**Malware**	Flask Modules; Incident; Dispatch	Malware runs in a Flask module and changes incident or dispatch records.	False incident status or unsafe service requests.	Backend validation; limited write access; human approval; monitoring.	TB2–TB3 / Flask / SQLite
T5	**Supply-chain Compromise**	Python Packages; Flask Application	A compromised Python package runs malicious code in the shared app.	Several modules may be affected; protected data may be changed.	Proposed package verification; pinned versions; controlled updates.	Flask Dependencies / TB2
T6	**Datacentre Failure**	Application Host; Simulated Nodes	Host or datacentre failure stops Flask/SQLite; simulation removes a node.	Real app outage; simulated replica capacity or quorum is reduced.	Protected backups; host recovery; simulated node recovery tests.	Host / SQLite / Simulated Regions
T7	**Network Partition**	Logical Consensus Nodes; Replica State	A simulated partition blocks messages between North, Central and South.	A subsystem loses quorum and must pause new protected commits.	Protocol-specific quorum checks; signed messages; safe recovery.	TB4 / Simulated Consensus
T8	**Data Corruption**	SQLite; Incident History; Audit Logs	Bad writes, app compromise or disk damage changes stored records.	Wrong resource decisions; incomplete or unreliable audit history.	Input checks; audit integrity checks; protected backups; recovery.	TB3 / SQLite / Audit

RESIDUAL RISKS — Privileged-account theft • malicious dependencies • Flask / SQLite host failure • prolonged quorum loss • data corruption

Design note: the three regions are logical simulation nodes, not real datacentres. Rate limiting, MFA, package checks and backups are proposed controls; test before claiming implementation.

Figure 2 : NERP/ERRS Architecture-Based Threat Model.

A3 — Architecture Assurance Summary

The NERP architecture supports confidentiality, integrity and availability through several security controls.

Confidentiality The authentication module checks user identities and sessions. The policy engine then uses ACM, Bell-LaPadula, Biba and context checks to control access. The two security domains separate operational records from restricted intelligence. The cross-domain gateway checks information before sharing it.

Integrity The system validates incident reports, and a dispatcher reviews assessment results before approving service requests. The policy engine controls protected actions, while audit records help staff track changes. Consensus nodes also check quorum before committing data. Integrity checks help detect damaged records.

AvailabilityThree simulated regions show how node failures and network partitions affect the system. The nodes can commit changes when the required quorum remains available. Staff can review incidents manually if automatic assessment fails. However, Flask and SQLite do not provide real multi-region failover. Long outages and host failures can still stop operations.

Part B – Consensus Mechanisms

B1. Protocol Selection

Subsystem	**Protocol**	**Reason for selection**
Regional Incident Database	Raft	Demonstrates leader based ordering and majority replication
Emergency Dispatch Service	Raft	Demonstrates coordinated updates through a leader

Resource Allocation Service	Paxos	Demonstrates agreement on one value while preserving accepted state
National Audit Log	BFT	Demonstrates agreement despite conflicting validator proposals
Intelligence Distribution Service	BFT	Demonstrates authenticated agreement among validators

The raft model is appropriate for the database and dispatch demonstrations because the model uses an elected leader to coordinate updates. Paxos could also provide agreement, but Raft offers an explicit leader and replication model.

Paxos is used for resource allocation because this focuses on preserving a previously accepted value across subsequent proposals. Its promise and accepted ballot model illustrates why an earlier agreement must not be silently replaced.

BFT is used for the audit and intelligence because these scenarios explicitly consider malicious validator messages. Raft and Paxos primarily address crash-failure models and do not independently provide the same Byzantine fault demonstration. These are educational design choices rather than experimentally proven performance comparisons.

All 3 mechanisms execute as Python simulations within a single application process.

B2. Consensus Mental Models

Raft – Leadership before consensus

The implementation in app/consensus/raft.py **it** uses three nodes: R1, R2 and R3. The candidate requires a majority of votes to become leader.. Example 2 votes here. The leader coordinates replication. An entry is committed only when a majority of nodes store it.

The network-partition experiment demonstrates that a node isolated from the majority cannot commit a new entry.

The election-storm experiment demonstrates a situation in which candidates repeatedly vote for themselves and fail to obtain a majority. The leadership-churn experiment demonstrates that an unreplicated local entry is not treated as committed history after leadership changes.

The simulation explains the importance of majority agreement, but does not implement real

network communication, persistent logs or randomised election timeouts.

Paxos – A majority is the system's memory

The implementation in app/consensus/paxos.py is going to use three acceptors. Each acceptor keeps track of the promises it has made , the values it has agreed to, and whether its data is available and safey stored .

A proposer must take approval from most of the acceptors before moving forward. If the acceptors report that they have already agreed to a value, the proposer must keep the value that was agreed tomost recently.

In the state loss test, the acceptors' stored information is cleared and marked as no longer reliable. When another proposal is made, the system returns SAFE_STOP. This shows that the system chooses to stop rather than risk making a new decision without knowing what was agreed on before.

However the durability flag is stored only in memory. This means it does not prove that the data would still be there if the computer lost power or the application was restarted.

BFT – Trust the protocol, not the participants

The BFT implementation in app/consensus/bft.py uses four validators and a quorum of three.

Validators authenticate messages using HMAC-SHA256. The verification process checks validator registration, active status, and key version and message authenticity. Conflicting authenticated proposals are not committed.

The equivocation experiment demonstrates that authentication alone does not establish agreement: a validator may authenticate conflicting values, but neither value is committed without the required quorum.
The implementation uses shared-secret HMAC rather than independent public-key digital signatures. It is therefore a simplified educational model.

B3. Consensus Assumptions

Protocol	**Main assumptions**
RAFT	Nodes vote according to the implemented term rules; majority needed to become leader; committed entries have majority replication
PAXOS	Acceptors preserve promises and accepted values; a majority is required; durable state remains trustworthy
BFT	Validators are registered; HMAC authentication is valid; key versions match; a quorum of three validators agrees..

The experiments test failures such as unregistered identities, revoked credentials, loss of acceptor state, missing majorities & network partitions.
However, the simulations do not establish independent validator administration, real network security, physical disk durability, public-key identities/independent datacentre failure domains.
These are external assumptions rather than demonstrated properties;

B4. Nine Consensus Experiments

The experiments are generated through run_experiment() in app/consensus/experiments.py.

The supplied evidence reports that all nine experiments passed their defined safety checks. In this context, PASS means that the specified faulty value was not committed/found.

ID	**FAULT**	OBSERVED RESULTS	FAULTY VALUE COMMITTED	RESULTS
BFT 1	Equivocation	DETECTED	NO	PASS
BFT 2	SYBIL IDENTITY	REJECTED	NO	PASS
BFT 3	KEY COMPROMISE	MITIGATED	NO	PASS
PAXOS 4	STATE LOSS	SAFE_STOP	NO	PASS
PAXOS 5	MAJORITY LOSS	NO_QUORUM	NO	PASS
PAXOS 6	QUORUM	REJECTED	NO	PASS

	MISCONFIGU RATION			
RAFT 7	NETWORK PARTITION	NO_QUARUM	NO	PASS
RAFT 8	ELECTION STORM	DEGRADED	NO	PASS
RAFT 9	LEADERSHIP CHURN	DEGRADED	NO	PASS

BFT-01 – Equivocation: V1 authenticates two different proposals for the same sequence. The conflicting values fail to get the required quorum, so neither commits.

BFT-02 – Sybil identity: An unregistered validator is rejected. The experiment demonstrates membership enforcement.

BFT-03 – Key compromise: V4 is revoked, its previous credential is rejected, and A replacement key version is a ccepted. The stale credential fails version validation.

PAXOS-04 – State loss: Previously accepted state is removed & marked non-durable. A subsequent proposal returns SAFE_STOP.

PAXOS-05 – Majority loss: Two acceptors become unavailable. The remaining acceptor cannot provide the needed majority.

PAXOS-06 – Quorum misconfiguration: A quorum of four is rejected when only three acceptors exist.

RAFT-07 – Network partition: The original value remains committed. An isolated leader cannot commit a new value u sing only one copy when two are required.

RAFT-08 – Election storm: Repeated self-voting prevents a candidate from obtaining a majority. The result is DEGRADED.

RAFT-09 – Leadership churn: An unreplicated local entry is not treated as committed when the original leader fails. A subsequent leader commits a confirmed value.

The JSON files in evidence/experiments/ provide the supporting experiment records. The results give the behaviour of the implemented simulations. They do not prove production distributed-consensus safety or availability.

B5. Consensus Hardening

Validator management is implemented in app/validators.py.

The mechanism supports authorised registration, rejection of unauthorised identities, revocation, authentication checks, key-version validation & credential replacement.

The test test_validator_admission_revocation_and_rotation verifies the principal admission & revocation behaviours.

the BFT3 experiment provides additional evidence of credential replacement and stale-key rejection.

An important limitation is that the validator field named public_key is used with shared secret HMAC authentication. It is not a public-key signature system. the prototype also lacks an external certificate authority and hardware backed key storage.

B6. Consensus Assurance

The nine experiments demonstrate that the simulations reject the defined faulty commitments under the tested conditions.

The evidence supports quorum enforcement,rejection of unauthorised validators,credential revocation,state-loss stopping and rejection of uncommitted entries.

However, all consensus state is held within one Python process. There are no independent nodes,

persistent consensus logs /genuine network communication. The implementation therefore demonstrates consensus security concepts, not a production-ready distributed platform.

Part C: Information Classification and Data Governance

C1. Classification Model

The prototype has four ordered levels of classification of confidentiality: PUBLIC, OPERATIONAL, SENSITIVE, and RESTRICTED. PUBLIC is for no sensitivity; OPERATIONAL for communications; SENSITIVE for medical or personal records; and RESTRICTED for intelligence and security information.

The thinking behind classification has each record take the highest classification assigned to its fields or classification hint. Caller names and telephone numbers are SENSITIVE because

this information exposure would (or could help) people gain private information, personally identify people, or contact people. Incident locations, casualty counts, dispatch units and timings serve emergency operations and are labelled OPERATIONAL. Audit actions and resource references also receive OPERATIONAL protection because they reveal internal activity.

SECRET and TOP SECRET hints map to RESTRICTED, CONFIDENTIAL to SENSITIVE, and OFFICIAL to OPERATIONAL. A substantive suspect identifier or CRITICAL/SEVERE threat level also receives RESTRICTED handling. Staff clearance metadata can therefore raise a responder record above the SENSITIVE level assigned to its name.

The model keeps operational fields while retaining sensitive fields for users with low clearance. This prototype mapping isn’t an official government classification equivalent. Confidentiality and integrity are still separate issues: security labels don’t guarantee that records are accurate, and unrestricted release still requires review.

C2. Classification Engine

The classage_record function in App/engines.py analyzes content and metadata by standardizing field names, identifying personal and operational fields, mapping category prompts and clearance levels, and checking if the text has configured keywords. The most applicable field label determines the record classification..

The result contains classification, a fields mapping, review_required and a generic reason. Review is requested for SENSITIVE or RESTRICTED records and for disagreements between a recognised hint and non-PUBLIC field labels. Placeholder suspect identifiers such as N/A do not independently trigger the RESTRICTED identifier rule. Unknown classification hints receive SENSITIVE field labels, although otherwise unrecognised content may fall back to PUBLIC.

Applying this engine to the supplied datasets yields 20 SENSITIVE incidents; eight SENSITIVE and 12 RESTRICTED responders; three OPERATIONAL and 17 RESTRICTED intelligence records; and 20 OPERATIONAL records each for dispatch and audit. Authorised API users receive classifications and derived views.

This method is explainable, but it can't reliably explain the context. Keyword rules might miss sensitive summaries or over-classify harmless wording. Review flags don't implement

an approval workflow. The importer stores initial classification metadata separately from runtime field tags. These statements need to be reconciled to avoid inconsistent handling.

C3 .Information Flow Engineering

The create_views function derives three views using field-level decisions without modifying the original records. It does not automatically declassify entire records.

The Operational View retains fields labelled PUBLIC or OPERATIONAL. This preserves incident identifiers, locations, types, priorities, casualty counts, agency assignments and dispatch details while excluding SENSITIVE caller names and telephone numbers. Dataset pages additionally filter visible fields against the signed-in role's clearance. The classification API is restricted to auditors, national commanders and intelligence analysts.

The analysis view provides category counts as well as lists of category, classification, and region values for each source record. Categories come from event type, threat level, audit action, or schedule status. Names, phone numbers, and suspect identifiers are omitted. However, the lists of categories and regions are not fully aggregated, which could reveal sensitive context.

The research view starts with operational fields and removes keys that match personal information patterns. Locations that include numbers are locally retained. Other fields like dates, timestamps, record IDs, and some link fields can be kept. For example, INC-001 keeps its event ID and the location in downtown Bristol. This is partial sanitization, not verified anonymization or approved for unrestricted external release.

The original JSON file remains unchanged; the source file's SHA-256 digest and audited dataset access support traceability. These mechanisms can't prove the accuracy of the initial data, nor can they make exported views tamper-proof. The retention assistant treats archiving and destruction as decisions, not file operations. Disclosure review, integrity protection, and lifecycle enforcement are still necessary.

C4. Retention and Governance

The retention_decision helper assigns demonstration periods of two years for PUBLIC, seven for OPERATIONAL, ten for SENSITIVE and twenty for RESTRICTED. Longer periods support investigations and accountability, while shorter periods reduce unnecessary storage and disclosure exposure. These are design assumptions, not statutory schedules; production policies must consider record purpose, applicable obligations and retention triggers.

The helper accepts a classification, legal_hold flag and age_years value. A legal hold takes priority and returns HOLD with deletion_status BLOCKED. Without a hold, records below their retention threshold return RETAIN. At or beyond the threshold, the helper returns DESTROY, archive_status ARCHIVED and deletion_status DESTROYED. For example, a three-year-old PUBLIC record receives DESTROY, whereas the same record under a hold receives HOLD.

When real-time events are closed, the application stores their assigned retention period and status in retention_records. Historical import datasets are kept separate from the real-time events table. The assistant doesn't calculate age from their timestamps, nor does it automatically enforce expiration across the five datasets..

These statuses indicate decisions: planned archive migration, secure deletion, key destruction, and backup deletion have not been implemented. The original files in the data/source remain intact. The prototype shows that the retention decisions are kept and priorities maintained; the schedule for operational execution and approval is still needed.

Part D :- ACCESS CONTROL AND SECURITY MODELS

D1. ACCESS CONTROL POLICY DESIGN

The ERRS prototype uses an Access Control Matrix to control permissions based on role for both resources and restricted resources. There are eight roles defined by their responsibilities in the emergency response environment.

The dispatcher has permission to manage incidents, validate or reject reports ask for information create service requests assign resources and read audit records. Police officers, paramedics and fire officers only have permission to respond to requests related to their service. Intelligence analysts can. Update intelligence records. Regional commanders have permission to access incidents in their region. National commanders have permission to view

incidents in all regions. Auditors have permission to read audit records, security policies, consensus experiments and governance data.

The permissions are specifically defined in the PERMISSIONS dictionary, in the app/security.py file. There is a has_permission() function that verifies whether the user, for their role can perform the requested operation. Further context checks limit access based on account status, operational purpose and regional assignment. The principle of privilege is applied so users only have permission to access information and perform operations needed to fulfil their roles. This minimises the risk of disclosure and limits the impact that could be caused by the compromise of a user account.

Role	**Main Authorised Activities**
Dispatcher	Incident management, verification, service requests, allocation and audit reading.
Police	Police service requests and responses
Paramedic	Ambulance service requests and responses
Fire	Fire service requests and responses
Intelligence Analyst	Intelligence reading and updating
Regional Commander	Regional incident access and escalation
National Commander	Wider incident access and escalation
Auditor	Read-only audit, policy, experiment and governance access

D2. BELL-LA PADULA CONFIDENTIALITY MODEL

The ERRS prototype uses the Bell–LaPadula (BLP) security model to keep information safe. Four classification levels are used: PUBLIC, OPERATIONAL, SENSITIVE and RESTRICTED.

The model relies on two principles: No Read Up and No Write Down. No Read Up stops users from reading information that's higher than their clearance level. No Write Down stops users from sending information to places that're lower than their clearance level.
These rules live in the `bell_lapadula()` function in `app/security.py`. The function checks the user's clearance against the classification of the requested resource. Reading is allowed when the user's clearance is the same as or higher than the resource classification. Writing is allowed when the user's clearance is the same as or lower than the resource classification.
For example a dispatcher with clearance cannot read RESTRICTED intelligence data. Likewise an intelligence analyst with RESTRICTED clearance cannot write directly to a PUBLIC resource.
These controls lower the risk of disclosure and leaks, between security domains.. The model needs correct classification labels and proper assignment of user clearances.

D3. BIBLA INTEGRITY MODEL

The ERRS prototype uses the Biba integrity model to keep information reliable and trustworthy. This is very important for incident reports, dispatch instructions, responder records and intelligence information because any unauthorised changes or bad data could mess up decisions.

The Biba integrity model uses two rules: No Read Down and No Write Up. No Read Down stops a higher-integrity subject from reading lower-integrity information. No Write Up stops a lower-integrity subject from changing higher-integrity information.

These rules are coded in the `biba()` function in `app/security.py`. For reading the subject's integrity level must not be higher than the resources level. For writing the subject's level must be equal to or higher than the resources level.

For example an intelligence analyst with RESTRICTED integrity cannot read a PUBLIC-integrity intelligence record under the Biba integrity model. Likewise a dispatcher, with integrity cannot change a RESTRICTED-integrity object.

These controls lower the chance that bad information will affect trusted records. Still the Biba integrity model needs integrity levels and trusted user identities.

D4. POLICY CONFLICTS AND TEST SCENARIOS

I see that the ERRS prototype blends the Access Control Matrix (ACM) Bell–LaPadula (BLP) Biba and contextual restrictions through the `evaluate_policy()` function in `app/security.py`. Access is granted only when all four checks allow the requested operation. If any check does not allow the final decision is DENY.

Three scenarios show how the combined policy works.

Scenario 1 – ACM permits but BLP denies: I notice a dispatcher asks for access to a RESTRICTED incident. The ACM says the dispatcher may read the incident. Bell–LaPadula says no because the dispatcher's OPERATIONAL clearance is lower than the incident's classification. This protects confidentiality.

Scenario 2 – BLP permits but Biba denies: I see an intelligence analyst asking for an intelligence record that has PUBLIC integrity. Bell–LaPadula says the analyst can read it because the analyst has clearance. Biba says no because the record's integrity level is lower than the analyst's level. This protects information integrity.

Scenario 3 – All policies permit: I know a dispatcher asks for an OPERATIONAL incident that's within their assigned region using an active account and the approved OPERATIONS purpose. The ACM, BLP, Biba and contextual checks all pass, giving a PERMIT.

These scenarios show that having permission to perform a task does not automatically mean access is justified. The combined policy engine balances confidentiality, integrity and operational needs, by allowing access while denying requests that break security rules.

D5. ACCESS - CONTROL ASSURANCE

The ERRS access control design tries to find a ground: keep data safe but don't get in the way when emergency services need quick access. The Access Control Matrix defines what each role can do. Bell-LaPadula and Biba help protect confidentiality and integrity. Then there are limits like account status, purpose and location that add another layer of control.

In the code this all comes together in evaluate_policy() in app/security.py. Access only happens if every single policy check passes. The test cases show this works. Unauthorized

users are blocked. People with the right role and purpose still get what they need. So having the right role isn't enough by itself. You still need to meet all the conditions.

The prototype has some limits. It depends on security and integrity levels. The contextual checks are pretty basic. Using one shared scale, for both confidentiality and integrity makes things simpler but not how things work in real life. It also doesn't show what a national-scale system would look like. Not every role or access combination is covered.

In all the design gives the ERRS prototype a strong security base. It brings together role-based permissions, confidentiality, integrity and context. If we're going to say it's ready for a national emergency response system we'd need testing. We'd also need authentication and more precise access controls.

Part E :- SECURITY GOVERNANCE AND ASSURANCE

E1. STRATEGIC SECURITY IMPACT ASSESSMENT

After NERP goes live, things might not always work as planned. New requests or unexpected problems could affect how the system works. The following five situations explain some of the issues that may come up.

1. Research organisation requests unrestricted intelligence access

Researchers may want to see all the intelligence stored in NERP. This could be a problem because some records contain private information about individuals, suspects and ongoing threats. Currently, anonymous users are not allowed to view RESTRICTED intelligence. This rule is there for a reason. If it is removed, people without permission could get hold of information they should not see.

2. Emergency responders request access to RESTRICTED intelligence

Imagine police officers dealing with a serious incident and needing information from NERP. Their OPERATIONAL clearance does not currently allow them to open RESTRICTED records. Giving them access without checking their authorisation would create another security risk. On the other hand, keeping the same restriction in every situation could slow

down their response. The system would need a way to handle genuine emergencies without giving everyone access to sensitive records.

3. A regional datacentre becomes permanently unavailable

A regional datacentre going offline could cause problems for other parts of NERP. Updates to incidents might stop, resource allocation could be affected, and communication between regions might become difficult. We tested a similar situation in the Raft simulation. When the majority of nodes are unavailable, the system stops accepting new commitments. This keeps the data consistent, but users may not be able to carry out certain operations. Our prototype tests this behaviour through simulation. It does not yet run on separate physical datacentres.

4. Validator credentials are compromised

If someone gets hold of a validator's credentials, they could try to use that account to perform operations they are not supposed to carry out. This could also affect the trustworthiness of audit records. In our BFT simulation, we tested how credentials could be revoked and replaced using key version checks. However, the system still relies on a shared HMAC secret. If that secret is stolen, an attacker could potentially misuse it to interfere with authentication.

5. New regulations require twenty year record retention

A requirement to keep records for twenty years would mean storing much more data than before. NERP would also need a way to organise and manage these records over such a long period. There is another concern here. Incident reports, dispatch details, intelligence records and audit logs may contain private information. Keeping them for longer also means that information remains at risk for a longer time. We have already demonstrated basic retention decisions and legal holds in the prototype. However, a complete system for long term storage and secure deletion has not yet been developed.

E2. SECURITY GOVERNANCE DECISIONS

The decisions below explain how I would handle each situation while keeping NERP secure and making sure emergency work is not affected unnecessarily.

1. Research organisation access

I would not allow researchers to access all intelligence records. Instead, they should receive only the information approved for research. Personal details and RESTRICTED information must remain protected. This means researchers may not get access to everything they need, but protecting sensitive information is more important. The research-view filtering already available in NERP can help with this. However, it cannot guarantee that every individual's identity will remain hidden.

2. Emergency responder access

I would keep the existing restrictions on RESTRICTED intelligence. Police officers should not be given direct access just because they are responding to an emergency. If they need this information, someone with the correct authority should approve the request. A second person should also check and approve it. This would help keep a record of who accessed the information and why. It might take longer during an emergency, but giving unrestricted access would be a bigger risk. The current prototype does not have this emergency approval feature yet.

3. Regional datacentre failure

If a regional datacentre fails, I would rather keep the data consistent than allow unconfirmed updates. The system should stop accepting protected commitments if it cannot reach the required majority. Other regions can still handle tasks that do not depend on the unavailable consensus. To bring the affected services back, NERP would need independent infrastructure and a properly tested disaster recovery plan. Some services may remain unavailable for a while, but this is safer than allowing different regions to store conflicting emergency records.

4. Validator credential compromise

If a validator's credentials are stolen, I would revoke them straight away. A new key version should then be issued, and the system should check the validator's credentials before allowing further operations. This would help protect the integrity of audit records. There could be a short disruption while the credentials are replaced. For a real deployment, I would also recommend public key authentication and secure key storage instead of relying only on the existing shared secret.

5. Twenty year record retention

I would follow the twenty year retention requirement and make sure access to stored records remains restricted. There may also be cases where records are needed for an ongoing investigation or legal process. Those records should be placed under a legal hold and must not be deleted while the hold is active. This will increase storage costs and require regular management. The current prototype shows how retention decisions can be made, but actual record deletion and a complete long term archiving process still need to be developed.

E3. NERP SECURITY ASSURANCE STATEMENT

The ERRS prototype includes several security features that were implemented and tested during development. These include user authentication, role based access control, Bell–LaPadula confidentiality, Biba integrity, information classification, audit verification and validator management.

The testing produced 17 passing application test records, nine consensus experiment records and five governance scenarios. These results show that the implemented features worked in the tests we carried out. However, passing these tests does not mean that the system is protected against every possible attack.

There are still some areas that need improvement. All regions currently use the same SQLite database, while the consensus nodes run as Python objects in memory. There is no actual network replication between separate machines. Validator authentication also uses shared secret HMAC instead of a full public key infrastructure. Staff accounts do not have multi factor authentication either. Another limitation is that filtering research data does not guarantee anonymity. The retention feature also does not carry out secure deletion of records.

The system's security also depends on a few important things being handled correctly. Information must be classified properly, user accounts must be trustworthy, credentials must remain protected and security policies must be configured correctly. These areas would need further checks before the system could be used in a real environment.

I would not recommend deploying the current ERRS prototype as national emergency response infrastructure. At this stage, it is better suited as an educational project that

demonstrates different security engineering principles. Before it could be considered for real world use, several improvements would be necessary. These include separate regional infrastructure, permanent consensus storage, stronger authentication, secure cryptographic key management and a properly tested disaster recovery process. An independent security assessment would also be needed before deployment.

Reference

1. Turoff, M., Chumer, M., Van de Walle, B. and Yao, X. (2004). 'The Design of a Dynamic EmergencyResponseManagement Information System (DERMIS)'. *Journal of Information Technology Theoryand Application*,5(4).

2. Van de Walle, B. and Turoff, M. (2008). 'Decision Support for Emergency Situations'. *InformationSystemsand E-Business Management*, 6, pp. 295–316.

3. Bell, D.E. and LaPadula, L.J. (1973). *Secure Computer Systems: Mathematical Foundations*.MITRECorporation.

4. Biba, K.J. (1977). *Integrity Considerations for Secure Computer Systems*. MITRE Corporation,ESD-TR-76-372.

5. Ongaro, D. and Ousterhout, J. (2014). 'In Search of an Understandable Consensus Algorithm'. *2014USENIXAnnual Technical Conference*.

6. Lamport, L. (2001). 'Paxos Made Simple'. *ACM SIGACT News*, 32(4), pp. 51–58.

7. Schneier, B. and Kelsey, J. (1999). 'Secure Audit Logs to Support Computer Forensics'. *ACM TransactionsonInformation and System Security*, 1(3), pp. 159–176.

8. Sweeney, L. (2002). 'k-Anonymity: A Model for Protecting Privacy'. *International Journal ofUncertainty,Fuzziness and Knowledge-Based Systems*, 10(5).

9. Michalec, O., Milyaeva, S. and Rashid, A. (2022). *Regulating digitisation of critical infrastructures: Recommendations to harmonise safety and security of Operational Technologies*. PolicyBristol Policy Briefing 120. University of Bristol. Available at: https://www.bristol.ac.uk/media-library/sites/policybristol/briefings-and-reports-pdfs/

2022/PolicyBristol_PolicyBriefing120_digitisation-critical-infrastructures_Michalec.pdf (Accessed: 5 November 2026).

10. University of Bristol (n.d.). *Security Infrastructure Suite: Cyber Range*. Bristol Digital Futures Institute. Available at: https://www.bristol.ac.uk/bristol-digital-futures-institute/facilities/security-infrastructure-suite/ (Accessed: 5 November 2026).

Appendix – AI Usage Declaration

Our group used AI tools, including ChatGPT, to a limited extent during this coursework. AI assistance was mainly used to understand code,debug implementation issues, clarify technical concepts and create a professional report structure and template. It was also used for initial research guidance on relevant academic papers,existing tools and related projects. The main technical research and understanding were developed through independent research and academic papers. All implementation, testing, experiments, analysis and final decisions were reviewed and completed by the group. We take full responsibility for the submitted work.